

On Using Large-Batches in Federated Learning

Sahil Tyagi

Indiana University Bloomington

ORCID

Abstract—Efficient Federated learning (FL) is crucial for training deep networks over devices with limited compute resources and bounded networks. With the advent of big data, devices either generate or collect multimodal data to train either generic or local-context aware networks, particularly when data privacy and locality is vital. FL algorithms generally trade-off between parallel and statistical performance, improving model quality at the cost of higher communication frequency, or vice versa. Under frequent synchronization settings, FL over a large cluster of devices may perform more work per-training iteration by processing a larger global batch-size, thus attaining considerable training speedup. However, this may result in poor test performance (i.e., low test loss or accuracy) due to generalization degradation issues associated with large-batch training. To address these challenges with large-batches, this work proposes our vision of exploiting the trade-offs between small and large-batch training, and explore new directions to enjoy both the parallel scaling of large-batches and good generalizability of small-batch training. For the same number of iterations, we observe that our proposed large-batch training technique attains about 32.33% and 3.74% higher test accuracy than small-batch training in ResNet50 and VGG11 models respectively.

Index Terms—Federated learning, Large batch training, Distributed systems, Neural networks, Deep learning

I. INTRODUCTION

Collaborative or *Federated learning (FL)* methods are optimized to perform on-device training when clients are resource-constrained [22], [23], communication latency and bandwidth is bounded [3], and data privacy or locality is paramount [1], [24]. These constraints arise from the explosive growth in big data and Internet-of-Things (IoT), as personal edge or mobile devices generate and collect data across various modalities, presenting new opportunities device-local, context-aware deep networks. For e.g., edge devices like NVIDIA AGX/Nano accelerate training/inference at the edge, while hardware-software co-design like Apple Intelligence runs contextual foundation models on proprietary phones and tablets [25]. FL with readily available resources over a large number of clients with varying data distributions present various challenges pertaining to training performance and speedup.

Different federated algorithms offer distinct trade-offs between parallel and statistical performance. Distributed training can be either *scaled-up* by batching more samples per iteration, or *scaled-out* by adding more clients to collaboratively train a globally shared model. Parallel performance or scaling is thus affected by factors like training topology, network latency and bandwidth, number of participating clients, batch-size, model-size and communication collective [3]. The statistical performance of deep neural networks (DNNs) is

influenced by its communication frequency, training hyperparameters (like learning-rate, batch-size, etc.) and data distribution, i.e., if training data across clients is independent and identically distributed (i.i.d.), unbalanced or skewed. As we see in §II, synchronization or aggregation frequency directly affects convergence quality; FL with higher communication volume produces models with higher accuracy (especially under non-i.i.d. settings), while synchronous training [8] attains the highest accuracy [2], [22]. However, synchronous training over a large number of clients may result in excessively large global batches that may attain high training accuracy, but low test accuracy/poor generalization [9], [11], [17].

Batch-size is a vital training parameter that affects both parallel and statistical performance of DNN training [5], [15]. Batching more samples on a client can leverage parallelism at an intra-device level (i.e., scaling-up), while inter-device aggregation of updates increases the effective, global batch-size (i.e., scaling-out) [8]. While the gradients computed over larger-batches are more representative of the true gradients (compared to full gradient descent) [5], [15], it results in lower model quality over test/unseen data compared to training over a smaller batch-size. In this paper, we address these trade-offs to propagate large-batch training in FL. At intra-device level, we develop a parallel performance model to estimate the compute cost of training DNNs at different batches for the given compute hardware, thus maximizing efficiency. We then cover prior art to improve statistical performance in large-batch training, and briefly introduce the notion of ‘gradient mapping’ using a teacher model to attain convergence quality over large-batches akin to small-batch training.

II. BACKGROUND AND RELATED WORK

In this section, we explore the trade-offs between parallel and statistical performance with different distributed training strategies, and explain how some techniques alleviate the generalization gap associated with large-batch training.

A. Parallel and Statistical Performance Trade-Offs in FL

FL may be centralized where clients updates communicate to a centralized parameter server or it may be decentralized (e.g., P2P, ring, etc.) [26] where updates are exchanged via collective communication [3]. Federated algorithms may use a low-frequency, high-volume communication strategy [26] or a high-frequency, low-volume approach [27]. *Federated averaging (FA)* [1] corresponds to the former, where updates are aggregated after certain training steps or epochs. Here, each

Table I. Convergence quality under BSP and FA communication models

Configuration	Partition	Comm.	Test Accuracy
ResNet50 CIFAR10	IID	BSP	82.54%
		FA	81.01%
	Non-IID	BSP	65.35%
		FA	58.72%
VGG11 CIFAR100	IID	BSP	83.91%
		FA	74.5%
	Non-IID	BSP	71.01%
		FA	32.86%
AlexNet CalTech101	IID	BSP	67.1%
		FA	62.67%
	Non-IID	BSP	66.6%
		FA	35.2%

client ‘ c ’ of C devices updates parameters w at iteration ‘ i ’ over loss function $\mathcal{L}(\cdot)$ and samples $b^{(c)}$ from distribution $\mathcal{B}^{(c)}$ by computing local rounds h after H global communication rounds. From Equation (1a), global model is aggregated over local models $w_{(i)+h}$ after every h local rounds.

$$w_{(i)+h+1}^{(c)} = w_{(i)+h}^{(c)} - \eta \odot \left(\frac{1}{b^{(c)}} \sum_{s \in \mathcal{B}_{(i)+h}^{(c)}} \nabla \mathcal{L}_s(w_{(i)+h}^{(c)}) \right) \quad (1a)$$

$$w_{(i)+1} := \frac{1}{C} \sum_{c=1}^C w_{(i)+H}^{(c)} \quad (1b)$$

Conversely, high-frequency, high-volume bulk synchronous-parallel (BSP) training aggregates updates across all clients at the end of *every* iteration. In Equation (1b), this corresponds to $H = 1$ so the global batch-size used to compute updates over each step is $B = \sum_{c=1}^C b^{(c)}$. Assuming the same local batch-size across clients, $B = |Cb|$, implying that BSP performs more work ‘for every iteration’ than local-SGD training.

Although federated algorithms have lower communication cost than BSP, the latter tends to achieve better statistical performance for a given training routine and data distribution. We see this in Table (I) where models trained over i.i.d. and non-i.i.d. datasets under BSP attain higher accuracy than federated averaging; non-i.i.d. datasets are partitioned with 1, 10 and 10 labels per-client across a cluster of 10 devices over CIFAR10, CIFAR100 and CalTech101 datasets. Updates are aggregated at every iteration in BSP, and once every training epoch in FA. *At the cost of higher synchronization overhead, BSP thus achieves better statistical performance than FA.*

In FA, the synchronization frequency is a key training parameter that determines overall speedup and convergence quality. Table (II) shows different models running i.i.d. and non-i.i.d. versions of various datasets in FA such that model is aggregated after every 250 and 1000 iterations respectively. We observe that the higher the synchronization frequency (i.e., smaller the FA steps), the better is the test accuracy. However, frequent synchronization incurs additional communication overhead, which may affect the cumulative training time.

Table II. Convergence in FA with different synchronization frequencies

Configuration	Partition	FA steps	Test Accuracy
ResNet50 CIFAR10	IID	250	78.07%
		1000	76.77%
	Non-IID	250	67.31%
		1000	66.26%
VGG11 CIFAR100	IID	250	75.96%
		1000	70.79%
	Non-IID	250	31.24%
		1000	12.0%
AlexNet CalTech101	IID	250	61.14%
		1000	59.62%
	Non-IID	250	23.8%
		1000	24.1%

B. Parallel-Scaling Considerations

Training can either be scaled-up (intra-device) or scaled-out (inter-device). The synchronization frequency in distributed settings affects the overall iteration time and consequently, parallel performance. During communication phase, cost is influenced by number of clients, model-size, device topology and collective implementation. For e.g., the bandwidth cost rises with the cluster-size in centralized star-topology, while decentralized ring-allreduce is bandwidth-optimal [3]. The process of forward and backpropagation at any step ‘ i ’ is identical from a computational standpoint: taking time t_c to compute model loss and gradients, t_{mov} for data-batching and movement overhead (e.g., between system memory and CPU, system memory and GPU, across memory hierarchies, etc.), and if applicable, communication overhead (Equation (2)).

$$t_{step}^{(i)} = t_c^{(i)} + t_{mov}^{(i)} + ((i + H) \% H == 0 ? 1 : 0) \cdot t_{sync}^{(i)} \quad (2)$$

In FL where models are synchronized after every $h \in [H]$ local steps across clients, so communication cost is zero in local-training phase and t_{sync} during synchronization phase. Thus, $|b|$ samples are used to train model on each client during local phase, and updates across $|Cb|$ samples are aggregated over the communication phase.

In BSP, training is *scaled-out* by keeping device batch-size fixed and adding more devices (i.e., weak-scaling). For $H = 1$ in Equation (2), updates are aggregated with communication overhead t_{sync} at every step. By incurring this cost, updates are effectively computed and applied over $|Cb|$ samples on every step (thus performing more work per-iteration by essentially using larger batches). On the other hand, training is *scaled-up* by increasing the batch-size on a device. Regardless of horizontal or vertical scaling, t_c and t_{mov} depend on a device’s training sample-size $|b|$. Both compute cost and data-movement overhead increases as device batch-size increases, as seen from their cumulative overhead in Figure (1). With larger batches, more data needs to be moved from disk/memory to devices’ compute cores, while computation cost rises from calculating additional activation maps in the forward pass. ViT uses smaller batches compared to other DNNs since the size of vision transformer model

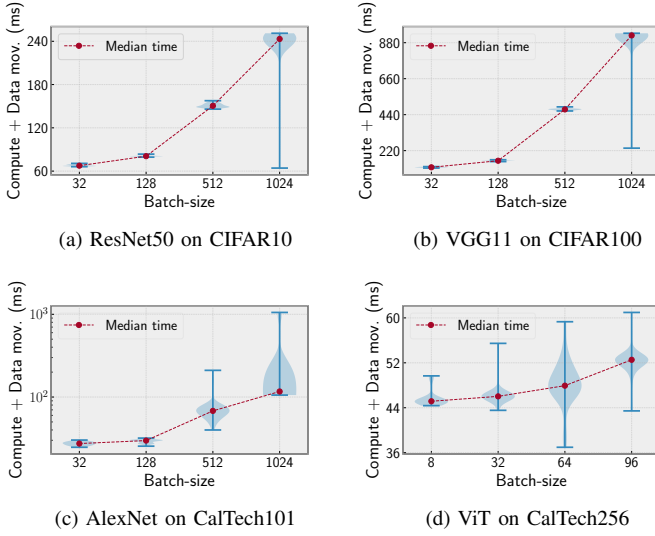


Fig. 1. Computation time and data-movement overhead rises as batch-size increases. The cumulative overhead rises sharply in some cases (ResNet50 and VGG11), and gradually in others (AlexNet and ViT), thus depends on the performance characteristics of a model and dataset configuration.

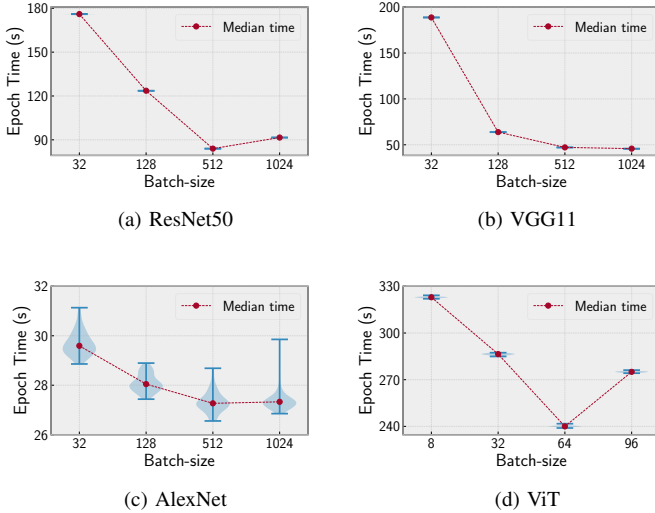


Fig. 2. Distribution of time taken to complete one training epoch over different batches. Training performance improves initially as data-movement overhead is more costly than computation cost, and total iterations decreases as B increases. However, compute cost outweighs as batch-size increases and performance eventually saturates.

grows proportionally to batch-size, thus bounded by limited device memory.

In spite of higher t_c and t_{mov} , larger batches tend to offer faster training performance. A model training over a dataset of size D with batch-size B takes iterations $I = \frac{D}{B}$ to complete a single epoch; training for E epochs thus takes $\frac{ED}{B}$ iterations. So, large batches (i.e., B) take fewer iterations to run over training data. The number of iterations directly affects the overall training time $T = I \cdot t_{step}$ [15]. Figure (2) shows the distribution and median time to complete a single epoch across

various DNNs. As B increases, per-epoch time decreases as well, until batches get excessively large. *Extremely large-batches have diminishing returns, as it cannot leverage intra-device parallelism any further and so compute cost can rise significantly for even marginal increase in client's batch-size.* We observe a higher epoch time at batch-size 1024 in ResNet50 and ViT, while the performance of VGG11 and AlexNet saturates at batch-size 512 over NVIDIA's K80 GPU.

C. Improving Convergence in Large-Batch Training

Although large-batches improve the parallel performance, it also results in worse statistical performance or generalization gap over small-batch training [9], [29]. Large batches tend to converge to sharp minima, while small-batches explore flatter minima [11]. [12] attributes this generalization gap with large-batches to a larger Hessian spectrum (commonly associated with sharp minima). We observed similar accuracy degradation in test performance of ResNet50, VGG11 and AlexNet in Figure (3) where larger batches like 512 and 1024 achieve lower test accuracy than smaller batches.

Prior art mitigates this issue with techniques like learning-rate warmup, batch and layer normalization, adaptive learning-rates and gradient clipping. To preserve SGD update (Equation (1a)), [6] scaled learning-rate proportional to the batch-size and trained ResNet-50 on ImageNet with a batch-size of 8000. [31] observed the optimal batch-size that maximized test performance to be proportional to learning-rate and size of training dataset. [20] reduced total training iterations by using a larger learning-rate and proportionally scaling up its batch-size. Post-local SGD [21] improved large-batch generalization by using BSP in the initial stages, followed by local-SGD. [7] injected additional noise in the gradients to improve large-batch generalization. Rather than inserting fixed Gaussian noise, [7] added perturbations in the form of diagonal Fisher noise. [10] performed extrapolation to smoothen optimization trajectory and avoid sharp minima. Here, gradients are computed at an extrapolated point, which is different from the current model state over which the SGD update is performed. LARS or layer-wise adaptive-rate scaling adjusts learning-rate on each layer based on the magnitude of the gradients proportional to its weights [4], [17], [18]. Inspired by LARS, LAMB [19] performed adaptive large-batch training with Adam optimizer and trained BERT in just over an hour. TVLARS [30] replaced learning-rate warmup of LARS with a sigmoid-type function in the early training phase. With elastic training, [5], [15] tune the global batch-size based on gradient noise. [16] looks at adaptive batch-sizes through the lens of second-order Hessian information and uses larger batches if the eigenvalue decays by a certain factor over training.

In FL, a device's resource availability and data distribution may vary over time. Thus, it is crucial to balance parallel and statistical performance aspect of large-batching training at both device-level and cluster-wide level for efficient FL.

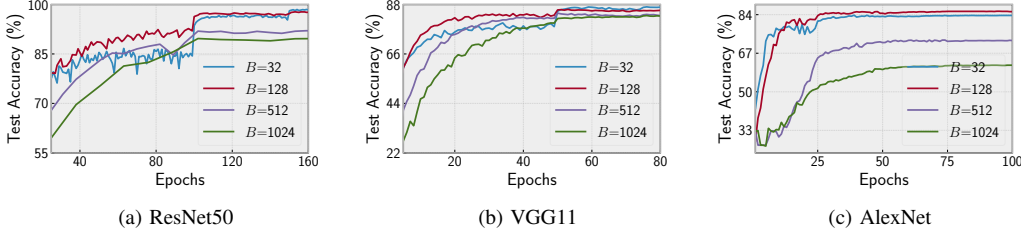


Fig. 3. Test accuracy is worse over larger batch-sizes due to convergence over sharper minima. When B increases, generalization improves as models converge over flatter minima. The generalization gap with large B corresponds to a larger Hessian spectrum associated with sharp minima.

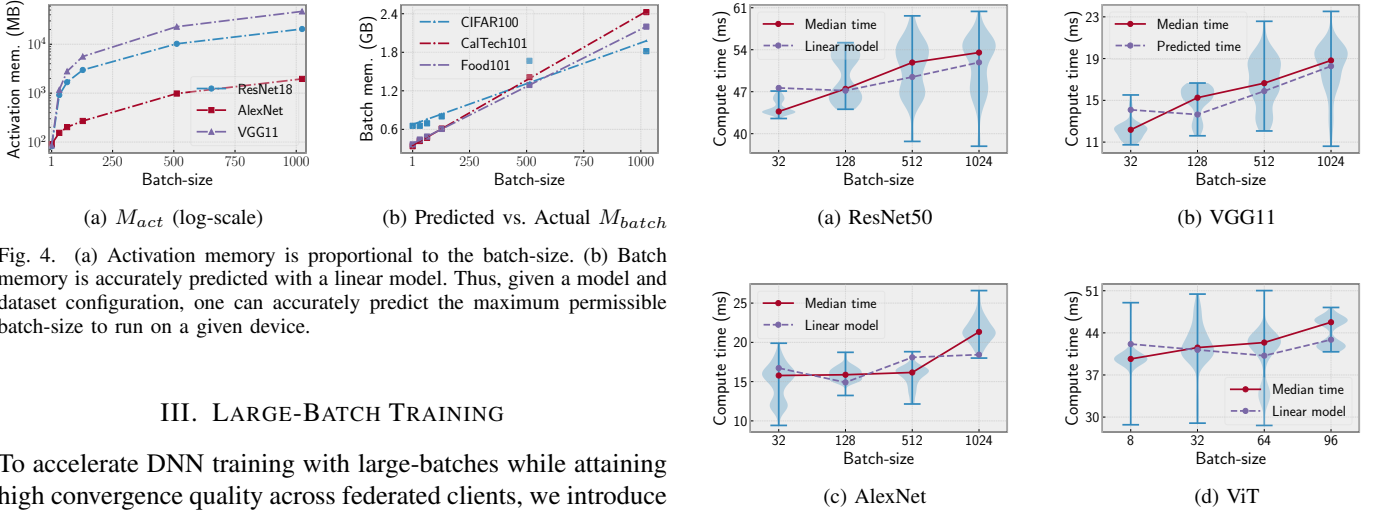


Fig. 4. (a) Activation memory is proportional to the batch-size. (b) Batch memory is accurately predicted with a linear model. Thus, given a model and dataset configuration, one can accurately predict the maximum permissible batch-size to run on a given device.

III. LARGE-BATCH TRAINING

To accelerate DNN training with large-batches while attaining high convergence quality across federated clients, we introduce a memory estimation model to predict the largest, usable batch-size given a client's computational resources. We then propose a parallel performance model to infer the batch-size that achieves maximum training speedup. Last, we give a broad overview of a statistical performance model that intelligently adds noise to large-batch updates to attain small-batch like generalizability.

A. Estimating Batch-size Bounds

The maximum permissible batch-size on a client is limited by its available resources. The total memory available on a device may limit the maximum batch-size that can be used without explicit checkpointing and recomputing intermediate model states [28]. The memory required for DNN training depends on storing the model parameters (M_{model}) and gradients (M_{grad}), type and state of optimizer M_{opt} (e.g., first or second-order), activation and batch memory (shown in Equation (3)).

$$M_{total} = M_{model} + M_{grad} + M_{opt} + M_{act} + M_{batch} \quad (3)$$

The last two vary with training batch-size and thus, limit the maximum permissible batch-size. M_{batch} corresponds to memory taken by the data-loading process to store and preprocess samples corresponding to the batch-size. For fully-connected and convolution networks, M_{act} is the total activa-

Fig. 5. Compute time of different models over varying batch-sizes. Compared to the median compute time, the linear model accurately predicts computation time within 9%, 15.7%, 13.6% and 6.34% error for ResNet50, VGG11, AlexNet and ViT respectively.

tion maps' memory computed on each sample in the forward pass. Figure (4a) shows the activation memory of ResNet18, VGG11 and AlexNet on the log-scale and shows that M_{act} rises linearly with the batch-size. We also plot the actual batch-memory for different batch-sizes (i.e., 1, 32, 64, 128, 512 and 1024) as scatter points in Figure (4b) and model the relationship between M_{batch} and batch-size as a linear regression problem. For CIFAR100, CalTech101 and Food101 datasets, the dashed lines show how the linear model predicts M_{batch} for different batches. CIFAR100 is predicted within 4.56-20.45% error-rate via linear fitting, while CalTech101 has a prediction error of only 0.45-3.47% and Food101 has an error range of 0.08-3.02%. By briefly running over extreme batch-sizes [15], we can thus accurately predict the memory needed for training and determine maximum batch-size for a given hardware.

From §II-B, we saw how the computation cost and data-movement overhead increased with larger batches. Figure (5) specifically shows the median and computation time distribution for different DNNs at varying batches. The trend is that

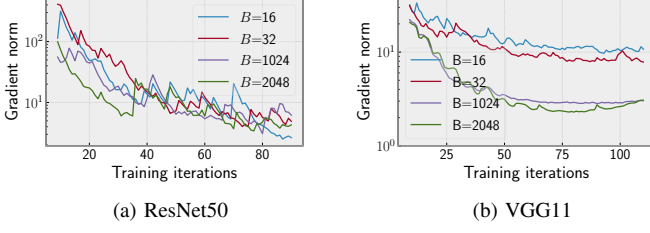


Fig. 6. Gradient norm in early training stages over different batches. As B increases, the signal-to-noise ratio of the gradients increases as well, thus updates get smaller and the gradient norm decreases.

compute cost increases with larger batches, as more processing is needed to be done to compute additional activation maps in the feedforward phase. Inspired by the parallel performance in [15], we model compute time with respect to batch-size, i.e., $t_c \propto |b|$. The dashed line shows the predicted time with a simplistic linear model fitted over the compute times logged for different batches (it does not look like a straight line as batch-sizes on the x-axis are not drawn to scale). Linear interpolation predicts the compute time for a given batch-size with an error margin of 0.6-9% in ResNet50, 2.88-15.7% in VGG11, 5.98-13.6% in AlexNet and 0.9-6.34% in ViT. *From these results, we observe that a linear model is able to predict the computation time for a given batch-size within good accuracy.* Accounting for even larger batches with synchronous data-parallel training can further be done by modeling communication cost for a given cluster and network configuration [15].

With the aforementioned memory and computation cost prediction models, parallel performance can accurately be inferred at both inter- and intra-device level, thus enabling large-batch training by determining the largest permissible batch-size, and the optimal batch-size with the least compute cost.

B. Improving Statistical Efficiency in Large-Batch FL

Training DNNs with large-batches results in updates that are smooth and precise, i.e., closer to the true gradient had we used full-batch gradient descent [5], [15]. However, this may lead to optimization towards sharp minima in the loss landscape, resulting in overfitting and poor generalization. On the other hand, gradients calculated over smaller-batches tend to be noisy due to high variance [5], which in turn provides additional regularization. This additional noise in smaller-batches helps DNNs escape sharp minima and converge to flatter minima, thus improving test performance and even adding robustness to small perturbations.

$$G^{(b_{small})} = G^{(b_{large})} + \gamma(b_{small}, b_{large}) \quad (4)$$

We see noisy gradients in small-batch training (compared to large-batches) in early training stages of ResNet50 and VGG11 in Figure (6). We plot the cumulative gradient norm for different batches and observe that large-batch (i.e., b_{large})

updates tend to be smaller than small-batches (b_{small}), differing by gradient noise γ , shown as Equation (4) [5], [15]. The noise term here corresponds to the variance between the updates computed over small and large batches. To improve the generalization gap in large-batch training, prior works have explored adding small perturbations or noise to gradient updates. This allows for more exploration over the local minima across devices, and eventually converge to flatter minima.

$$G_{(i)}^{(b_{large})} = \frac{1}{|B^{(b_{large})}|} \nabla \mathcal{L}(w_{(i)}, B^{(b_{large})}) \quad (5a)$$

$$\tilde{G}_{(i)} = \mathcal{A}(G_{(i)}^{(b_{large})}) \quad (5b)$$

$$w_{(i+1)} = w_{(i)} - \eta \odot \tilde{G}_{(i)} \quad (5c)$$

Rather than adding fixed noise at every iteration or compute the Fisher noise matrix [7], we propose an alternate approach highlighted in Equation (5). Instead of using large-batch gradients $G_{(i)}^{(b_{large})}$ at iteration ‘ i ’ to apply gradient descent update on model parameters, we use a pretrained encoder-like teacher model $\mathcal{A}(\cdot)$ that maps large-batch updates to an *approximated* small-batch $\tilde{G}_{(i)}$. The estimated gradient $\tilde{G}_{(i)}$ is then used to update model state for the next iteration (Equation (5c)). The teacher model is “briefly” trained by deploying the student (i.e., the DNN to be trained with larger batches) with a small and a large batch-size respectively ($B^{(b_{small})}$ and $B^{(b_{large})}$). Then, $\mathcal{A}(\cdot)$ is trained with gradients from $B^{(b_{large})}$ as input and generating gradients corresponding to $B^{(b_{small})}$ as the output. We refer to $\mathcal{A}$ as a teacher and original DNN as the student since we want to learn the mapping from small to large-batch updates from $\mathcal{A}$. The primary overhead with this approach comes from training $\mathcal{A}$; so the cost will be low if the teacher is trained briefly for a few iterations/epochs, and vice versa. *We don’t necessarily need the teacher model to be highly accurate in predicting small-batch gradients; it simply needs to be able to map the variance between large and small-batch updates within reasonable estimates.* Throughout training, the scale and trajectory of gradients may vary, as attributed to the initial training phase [33] and critical periods [32]. Updating and retraining the teacher model in such phases may be required for accurate small-batch estimates. Prior works have explored using autoencoders for adaptive gradient compression [13] and privacy-preserving FL [14], but not for large-batch training.

$$\mathcal{U}(X) = \begin{cases} 1.0 & \text{in critical training phases} \\ X & \text{otherwise} \end{cases} \quad (6a)$$

$$\tilde{G}_{(i)} = \mathcal{U}(X, G_{(i)}^{(b)}) \quad (6b)$$

$$\tilde{G}_{(i)} = \begin{cases} G_{(i)}^{(b)} & \text{in critical training phases} \\ X \odot G_{(i)}^{(b)} & \text{otherwise} \end{cases} \quad (6c)$$

Naïve approach. We assess the efficacy of this technique by substituting $\mathcal{A}(\cdot)$ with a simplistic step function $\mathcal{U}(\cdot)$ in our

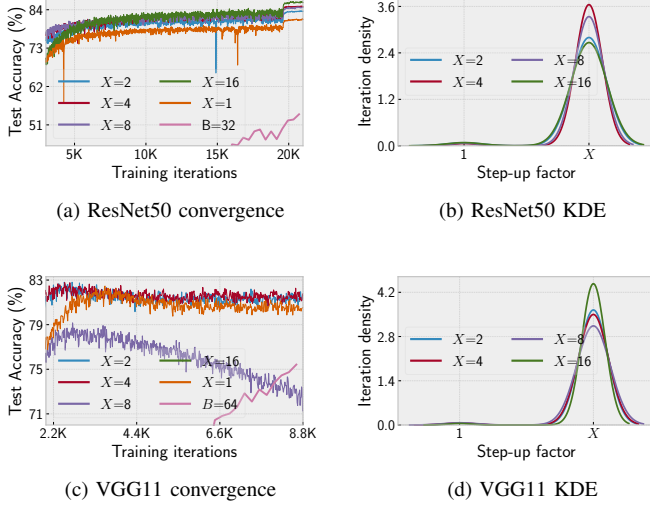


Fig. 7. Convergence quality and iteration kernel density estimates (KDE) when scaling-up the step function for various X values at gradient change threshold 0.5. Model quality improves over the baseline large-batch training (i.e., $X=1$) in most cases, but degrades in others when X becomes too large (e.g. VGG11 with $X=8$ and $X=16$).

preliminary evaluation. From Equation (6a), the step function steps-up or increases the gradients by factor X when training is not in critical regions, and steps-down to the original, computed gradients under sensitive training phases. Replacing $\mathcal{A}(\cdot)$ in Equation (5b) with $\mathcal{U}(\cdot)$, we model gradients computed over larger batches as illustrated in Equations (6b) and (6c). Thus, we use the original gradients computed over the chosen batch-size b in sensitive regions, and scale up the gradients by X as if they were computed from a smaller batch-size. This is in line with Equation (4), where we set γ to 0 in critical phases and inject artificial noise equivalent to $(X-1)G^{(b_{large})}$ otherwise (as shown in Equation (7)).

$$G^{(b_{small})} \approx G^{(b_{large})} + (X-1)G^{(b_{large})} = X \odot G^{(b_{large})} \quad (7)$$

Thus, it is important to detect sensitive training phases in order to test this strategy. From [2], critical phase can be detected from changes in gradient norm over the iterations, as shown by the gradient change metric $\Delta(G_{(i)}^{(b)})$ in Equation (8).

$$\Delta(G_{(i)}^{(b)}) = \left| \frac{|G_{(i)}^{(b)}|^2 - |G_{(i-1)}^{(b)}|^2}{|G_{(i-1)}^{(b)}|^2} \right| \quad (8)$$

We train ResNet50 and VGG11, each with batch-size 1024 and step-up the computed gradients by setting X to $2\times$, $4\times$, $8\times$ and $16\times$ respectively. In sensitive regions, we step-down the gradients by setting the step function to $1\times$ (from Equation (6c)). From [2], we set a threshold of 0.5 on the rate of gradient change, so $\Delta(G_{(i)}^{(b)}) \geq 0.5$ steps-up the gradients to the chosen X , while updates are stepped down to $1\times$ when $\Delta(G_{(i)}^{(b)}) < 0.5$.

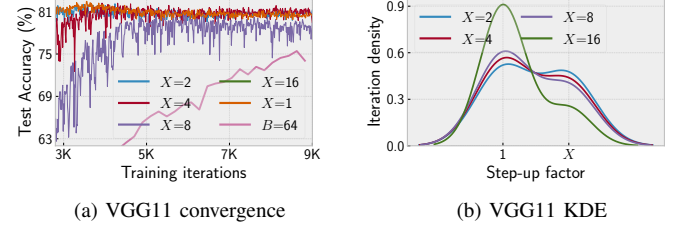


Fig. 8. VGG11 convergence at various step-up factors with a higher gradient change threshold 0.8. Compared to the lower threshold (from Figure (7)), model quality improves at $X=8$ by stepping down to 1.0 for most iterations, as seen from the density plots. However, VGG11 still failed to converge at $X=16$ as the scale-up factor was very large and diverged the model.

We show the results in Figure (7) for different step-up factors along with $X=1$ (which corresponds to baseline training with a large batch-size 1024). From Figure (7a), we see that the convergence quality of ResNet50 increases with X . Here, $X=16$ attains higher accuracy than other step-up factors, and even more than the baseline configuration. Thus, naively scaling up the gradient updates improves model quality and diminishes generalization gap even at larger batches. Most of the iterations use scaling factor X instead of $1\times$ for $\Delta(G_{(i)}^{(b)}) \geq 0.5$, as seen from the kernel density estimates or KDEs of training iterations. The KDE for $X=1$ is omitted since the original gradients are used to update model parameters and there is no additional step-up in this configuration (so only $1\times$). Thus, we see that even though majority of the updates scale-up the gradients, model quality improves even with higher scaling factors (i.e., X). For reference with regard to small-batch training, Figure (7a) also plots the accuracy curve for batch-size 32, which shows that large-batch training is faster to converge in the same iterations, especially with gradient scaling. Compared to $B=32$, gradient scale-up with a large batch-size attained up to 32.33% more accuracy.

However, convergence in VGG11 improved over the baseline only for $X=2$ and $X=4$, and degraded for larger X , as seen in Figure (7c). The model loses considerable accuracy over $X=8$ (compared to $X=1$), and fails to improve beyond 10% for $X=16$ (thus omitted from the figure). From Figure (7d), most of the training iterations use scaling-factor X (over $1\times$) throughout the training phase, so larger stepping factors are detrimental to convergence quality in this case.

To improve the convergence quality in VGG11, we vary the sensitivity of the $\mathcal{U}(\cdot)$ function by raising the gradient change threshold up to 0.8. Thus, the step function is more censorious to model updates and uses the original gradients in majority of the iterations (i.e., $1\times$). The larger step-up factor X is used only in highly sensitive iterations due to the higher $\Delta(G_{(i)}^{(b)})$ threshold. Although $X=8$ configuration still underperformed $X \in [1, 2, 4]$, we see that compared to the lower threshold of 0.5 in Figure (7c), model accuracy significantly improved over the larger $X=8$ in Figure (8a). Even though $X=16$ still failed to achieve similar accuracy levels (thus, omitted from the results), it still attained higher accuracy of $\approx 30\%$ (compared

to only 10% for $X=16$ at threshold 0.5). From Figure (8b), the improvement in convergence quality can be attributed to intelligently switching between factors $1\times$ and X based on critical phase identification with a higher gradient change threshold. Configurations $X \in [2,4,8]$ with the larger batch-size 1024 outperformed small-batch training with $B=64$ by attaining better accuracy in the same number of iterations.

Summary: *With the aforementioned naive approach using a simple, static step-up function, some models and configurations perform better than baseline large-batch training, while others degrade a model's statistical performance (test loss or accuracy). We observed the latter for larger X in VGG11 model, and the former for all X values in ResNet50 and smaller X in VGG11. Intuitively switching between the original and scaled updates based on critical phase detection is crucial for model convergence, where such sensitive periods are successfully identified with the gradient change metric. However, as seen from the above results, the ideal threshold can vary with model and configuration, and thus requires carefully choosing the optimal value for this threshold. We conjecture that replacing the stepping function $\mathcal{U}(\cdot)$ with a knowledge distilling teacher model $\mathcal{A}(\cdot)$ that maps large-batch gradients to their small-batch equivalent would improve the quality of estimated gradients even further, thus providing training speedup of large-batches as well as robust generalization commonly associated with small-batches.*

Gradient compression and large-batch training: Frequent communication can become the performance bottleneck in synchronous training, especially when the model-size and cluster of federated devices is large. Gradient compression techniques like sparsification, quantization or low-rank approximations [37] reduce the communication volume, thus accelerating the training process [27]. In Equation (9a), a compression operator $\mathcal{C}(\cdot)$ reduces gradients $G^{(b)}$ computed over batch-size b to compressed update $G^{(*)}$.

$$G^{(*)} = \mathcal{C}(G^{(b)}) \quad (9a)$$

$$G^{(b)} = G^{(*)} + (G^{(b)} - G^{(*)}) \quad (9b)$$

$$G^{(b)} = G^{(*)} + \phi(b) \quad (9c)$$

Prior work [34] has explored the correlation between batch-sizes and compression during critical regimes by assuming the stochastic gradient $G^{(b)}$ to be composed of a sparse mean $G^{(*)}$ and a dense noise (i.e., $G^{(b)} - G^{(*)}$) for models trained with sparsity inducing norms, shown in Equation (9b). Gradients computed over large-batches correspond to highly compressed updates, while small-batch training corresponds to weakly compressed gradients. The backpropagated gradients $G^{(b)}$ can thus be decomposed into sparse, compressed updates $G^{(*)}$ and dense noise $\phi(b)$. The relation between batch-size and gradient compression is further corroborated by the analogy between Equations (4) and (9c). Thus, using large-batches in federated learning makes DNN training more robust to high degree of compression and provides considerable communication savings, resulting in good overall speedup.

Heterogeneity under federated learning: Heterogeneity is pervasive across edge, cloud and high-performance computing. The devices in a federated cluster may have inherent variance in their respective computational capability, or partial resource availability from concurrent execution of multiple jobs. Heterogeneity may also rise from the data volume processed by each device. To handle compute heterogeneity, [35], [36] proposed variable and dynamic-batching where each device is allocated a batch-size proportional to its compute capability. To address data volume imbalance, prior works have explored weighted aggregation by scaling updates proportional to the number of samples processed by each device [1], [22], [35]. In both resource and data volume heterogeneity, batches on some devices can become extremely small, and excessively large on others. Using a teacher model as described in Equation (5) may also help modulate the disproportional updates computed from variable/heterogeneous training samples across devices.

IV. CONCLUSION AND FUTURE WORK

It is crucial to consider both parallel and statistical efficiency in large-batch training for fast, scalable and accurate FL systems. In this work, we first introduce a parallel performance model to estimate the optimal batch-size that minimizes training time, and determine the largest, permissible batch-size usable on a given hardware. Using a larger batch-size for training can improve the execution performance of federated learning in two ways: one, by lowering cumulative iterations over the epochs and reducing the data-movement overhead, and second, by allowing higher compression factors in compression-enabled distributed training.

To improve the generalization gap with large-batches, we envision an encoder-based teacher-student model configuration for knowledge distillation where the teacher would output small-batch updates from the large-batch gradients generated by the student model. We additionally show a simple step function (as a naive substitute to the encoder-based model) that either scales-up or scales-down the gradients by a fixed value under certain training periods can significantly improve the convergence quality over vanilla large-batch training. Compared to small-batch settings, we also observed higher accuracy over the same set of training iterations. In our preliminary evaluation, gradients scaled to $16\times$ in less sensitive regions and $1\times$ (i.e., the original updates) under critical phases attained up to 4.89% better accuracy than the baseline (corresponding to large-batch training with $X=1$ at batch-size 1024) and 32.33% higher accuracy than small-batch training (with batch-size 32) in ResNet50. In VGG11, although the naive step function at $8\times$ scale-up attains about 3.3% lower accuracy than baseline, but still achieved 3.74% more accuracy than small-batching training for the same iterations.

In future work, we intend to explore different architectures like transformer encoders and autoencoders for the proposed teacher model. Since the objective of teacher-model is to learn the mapping between updates computed over a large and small batch-size, another challenge is to determine how large or

small the chosen batches should be. Based on prior work, we will study gradient noise to determine the optimal batches to be used. Last, we will focus on using adaptive batch-sizes in conjunction with gradient compression in order to attain high model quality with minimal communication overhead.

REFERENCES

- [1] McMahan, H. B. et al. "Communication-Efficient Learning of Deep Networks from Decentralized Data." International Conference on Artificial Intelligence and Statistics (2016).
- [2] Tyagi, Sahil and Martin Swamy. "Accelerating Distributed ML Training via Selective Synchronization." 2023 IEEE International Conference on Cluster Computing (CLUSTER) (2023): 1-12.
- [3] Tyagi, Sahil and Martin Swamy. "Flexible Communication for Optimal Distributed Learning over Unpredictable Networks." 2023 IEEE International Conference on Big Data (BigData) (2023): 925-935.
- [4] You, Yang et al. "ImageNet Training in Minutes." Proceedings of the 47th International Conference on Parallel Processing (2017).
- [5] McCandlish, Sam et al. "An Empirical Model of Large-Batch Training." ArXiv abs/1812.06162 (2018).
- [6] Goyal, Priya et al. "Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour." ArXiv abs/1706.02677 (2017).
- [7] Wen, Yeming et al. "An Empirical Study of Large-Batch Stochastic Gradient Descent with Structured Covariance Noise." 23rd International Conference on Artificial Intelligence and Statistics (AISTATS) (2020).
- [8] Das, Dipankar et al. "Distributed Deep Learning Using Synchronous Stochastic Gradient Descent." ArXiv abs/1602.06709 (2016).
- [9] Hoffer, Elad et al. "Train longer, generalize better: closing the generalization gap in large batch training of neural networks." 31st International Conference on Neural Information Processing Systems (NIPS) (2017).
- [10] Lin, Tao et al. "Extrapolation for Large-batch Training in Deep Learning." 37th Proceedings of Machine Learning Research (PMLR) (2020).
- [11] Keskar, Nitish Shirish et al. "On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima." Proceedings of International Conference on Learning Representations (ICLR) (2017).
- [12] Yao, Zhewei et al. "Hessian-based Analysis of Large Batch Training and Robustness to Adversaries." 32nd International Conference on Neural Information Processing Systems (NeurIPS) (2018).
- [13] Abrahamyan, Lusine et al. "Learned Gradient Compression for Distributed Deep Learning." IEEE Transactions on Neural Networks and Learning Systems 33 (2021): 7330-7344.
- [14] Chen, Yiming et al. "Learned Parameter Compression for Efficient and Privacy-Preserving Federated Learning." IEEE Open Journal of the Communications Society 5 (2024): 3503-3516.
- [15] Tyagi, Sahil and Prateek Sharma. "Scavenger: A Cloud Service For Optimizing Cost and Performance of ML Training." 2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid): 403-413.
- [16] Yao, Zhewei et al. "Large batch size training of neural networks with adversarial training and second-order information." ArXiv abs/1810.01021.
- [17] You, Yang et al. "Large Batch Training of Convolutional Networks." arXiv: Computer Vision and Pattern Recognition (CVPR) (2017).
- [18] Jia, Xianyan et al. "Highly Scalable Deep Learning Training System with Mixed-Precision: Training ImageNet in Four Minutes." Systems for ML workshop (NeurIPS) (2018).
- [19] You, Yang et al. "Large Batch Optimization for Deep Learning: Training BERT in 76 minutes." Proceedings of the 8th International Conference on Learning Representations (ICLR) (2020).
- [20] Smith, Samuel L. et al. "Don't Decay the Learning Rate, Increase the Batch Size." Proceedings of the 6th International Conference on Learning Representations (ICLR) (2018).
- [21] Lin, Tao et al. "Don't Use Large Mini-Batches, Use Local SGD." Proceedings of the 8th International Conference on Learning Representations (ICLR) (2020).
- [22] Tyagi, Sahil and Martin Swamy. "ScaDLES: Scalable Deep Learning over Streaming data at the Edge." IEEE International Conference on Big Data (Big Data) (2022): 2113-2122.
- [23] Wang, Shiqiang et al. "Adaptive Federated Learning in Resource Constrained Edge Computing Systems." IEEE Journal on Selected Areas in Communications 37 (2018): 1205-1221.
- [24] Li, Tian et al. "Federated Learning: Challenges, Methods, and Future Directions." IEEE Signal Processing Magazine 37 (2019): 50-60.
- [25] Gunter, Tom et al. "Apple Intelligence Foundation Language Models".
- [26] Yuan, Liangqi et al. "Decentralized Federated Learning: A Survey and Perspective." ArXiv abs/2306.01603 (2023).
- [27] Tyagi, Sahil and Martin Swamy. "GraVAC: Adaptive Compression for Communication-Efficient Distributed DL Training." 2023 IEEE 16th International Conference on Cloud Computing (CLOUD): 319-329.
- [28] Rajbhandari, Samyam et al. "ZeRO: Memory Optimization Towards Training A Trillion Parameter Models." ArXiv abs/1910.02054 (2019).
- [29] Golmant, Noah et al. "On the Computational Inefficiency of Large Batch Sizes for Stochastic Gradient Descent." ArXiv abs/1811.12941 (2018).
- [30] Do, Khoi Duy et al. "Revisiting LARS for Large Batch Training Generalization of Neural Networks." ArXiv abs/2309.14053 (2023).
- [31] Smith, Samuel L. et al. "A Bayesian Perspective on Generalization and Stochastic Gradient Descent." ICLR 2018.
- [32] Achille, Alessandro et al. "Critical Learning Periods in Deep Neural Networks." ArXiv abs/1711.08856 (2017).
- [33] Frankle, Jonathan et al. "The Early Phase of Neural Network Training." ArXiv abs/2002.10365 (2020).
- [34] Agarwal, Saurabh et al. "Accordion: Adaptive Gradient Communication via Critical Learning Regime Identification." 4th MLSys (2021).
- [35] Tyagi, Sahil and Prateek Sharma. "Taming Resource Heterogeneity In Distributed ML Training With Dynamic Batching." 2020 IEEE ACSOS.
- [36] Tyagi, Sahil and Prateek Sharma. "OmniLearn: A Framework for Distributed Deep Learning Over Heterogeneous Clusters." IEEE Transactions on Parallel and Distributed Systems 36 (2025): 1253-1267.
- [37] Tyagi, Sahil. "An Overview of Computational and Communication Mechanisms for Scalable AI Systems". 2025 ResearchGate DOI:10.13140/RG.2.2.21481.61283.